

PYTHON FUNCTIONS FOR BEGINNERS

– Cách dùng chi tiết

1. Hàm cơ bản

Hàm	Công dụng	Ví dụ
<code>print()</code>	In giá trị ra màn hình	<code>print("Xin chào")</code>
<code>type()</code>	Kiểm tra kiểu dữ liệu	<code>type(5) → int</code>
<code>id()</code>	Lấy địa chỉ bộ nhớ của biến	<code>id(x)</code>
<code>exit()</code>	Thoát chương trình	<code>exit()</code>
<code>input()</code>	Nhận dữ liệu người dùng nhập	<code>name = input("Tên: ")</code>
<code>len()</code>	Đếm số phần tử/độ dài	<code>len("hello") → 5</code>
<code>help()</code>	Xem tài liệu hướng dẫn của hàm	<code>help(print)</code>

2. Hàm số học

Hàm	Công dụng	Ví dụ
<code>abs()</code>	Lấy giá trị tuyệt đối	<code>abs(-5) → 5</code>

Hàm	Công dụng	Ví dụ
<code>round()</code>	Làm tròn số	<code>round(3.567, 2) → 3.57</code>
<code>int()</code>	Chuyển sang số nguyên	<code>int("10") → 10</code>
<code>float()</code>	Chuyển sang số thực	<code>float("3.14") → 3.14</code>
<code>min()</code>	Tìm giá trị nhỏ nhất	<code>min(3, 7, 1) → 1</code>
<code>max()</code>	Tìm giá trị lớn nhất	<code>max(3, 7, 1) → 7</code>
<code>pow()</code>	Tính lũy thừa	<code>pow(2, 3) → 8</code>
<code>divmod()</code>	Trả về (thương, dư)	<code>divmod(7, 2) → (3, 1)</code>

3. Hàm dãy (sequence)

Hàm	Công dụng	Ví dụ
<code>list()</code>	Tạo danh sách	<code>list("abc") → ['a', 'b', 'c']</code>
<code>tuple()</code>	Tạo tuple (bất biến)	<code>tuple([1,2,3]) → (1,2,3)</code>
<code>range()</code>	Tạo dãy số	<code>range(0,5) → 0,1,2,3,4</code>

Hàm	Công dụng	Ví dụ
<code>sorted()</code>	Sắp xếp	<code>sorted([3,1,2]) → [1,2,3]</code>
<code>reversed()</code>	Đảo ngược thứ tự	<code>list(reversed([1,2,3])) → [3,2,1]</code>
<code>enumerate()</code>	Đánh số kèm phần tử	<code>list(enumerate(['a','b'])) → [(0,'a'),(1,'b')]</code>
<code>map()</code>	Áp dụng hàm cho từng phần tử	<code>list(map(str, [1,2])) → ['1','2']</code>
<code>filter()</code>	Lọc phần tử theo điều kiện	<code>list(filter(lambda x: x>2, [1,2,3])) → [3]</code>
<code>slice()</code>	Cắt một đoạn dãy	<code>a[slice(1,3)]</code>

4. Hàm xử lý chuỗi (string)

Hàm	Công dụng	Ví dụ
<code>str()</code>	Chuyển sang chuỗi	<code>str(10) → "10"</code>

Hàm	Công dụng	Ví dụ
<code>chr()</code>	Số → ký tự Unicode	<code>chr(65) → "A"</code>
<code>format()</code>	Định dạng chuỗi	<code>"{}-{}".format(1,2) → "1-2"</code>
<code>find()</code>	Tìm vị trí chuỗi con (trả -1 nếu không có)	<code>"hello".find("l") → 2</code>
<code>index()</code>	Tìm vị trí (báo lỗi nếu không có)	<code>"hello".index("l") → 2</code>
<code>replace()</code>	Thay thế ký tự/chuỗi	<code>"hi".replace("h", "H") → "Hi"</code>

5. Hàm tập hợp / collection

Hàm	Công dụng	Ví dụ
<code>frozenset()</code>	Tạo set bất biến	<code>frozenset([1,2,3])</code>
<code>dict()</code>	Tạo từ điển (key-value)	<code>dict(a=1, b=2) → {'a':1, 'b':2}</code>
<code>bool()</code>	Chuyển sang True/False	<code>bool(0) → False</code>
<code>set()</code>	Tạo tập hợp (không trùng lặp)	<code>set([1,1,2]) → {1,2}</code>

6. Hàm lặp (iterable)

Hàm	Công dụng	Ví dụ
<code>iter()</code>	Tạo iterator	<code>it = iter([1,2,3])</code>
<code>next()</code>	Lấy phần tử tiếp theo	<code>next(it)</code>
<code>zip()</code>	Ghép nhiều dãy lại	<code>list(zip([1,2],['a','b']))</code> → <code>[(1,'a'),(2,'b')]</code>
<code>all()</code>	True nếu tất cả phần tử đều True	<code>all([True, True])</code> → True
<code>any()</code>	True nếu có ít nhất 1 phần tử True	<code>any([False, True])</code> → True

7. Toán & băm nâng cao

Hàm	Công dụng	Ví dụ
<code>hash()</code>	Tạo mã băm của đối tượng	<code>hash("abc")</code>

8. Làm việc với file

Hàm	Công dụng	Ví dụ
<code>open()</code>	Mở file	<code>f = open("a.txt", "r")</code>
<code>readline()</code>	Đọc 1 dòng	<code>f.readline()</code>

Hàm	Công dụng	Ví dụ
<code>readlines()</code>	Đọc tất cả dòng, trả về list	<code>f.readlines()</code>
<code>write()</code>	Ghi chuỗi vào file	<code>f.write("hi")</code>
<code>writelines()</code>	Ghi nhiều dòng	<code>f.writelines(["a\n", "b\n"])</code>
<code>close()</code>	Đóng file	<code>f.close()</code>
<code>seek()</code>	Di chuyển con trỏ đọc/ghi	<code>f.seek(0)</code>
<code>tell()</code>	Vị trí hiện tại của con trỏ	<code>f.tell()</code>
<code>remove()</code>	Xóa file (thuộc module <code>os</code>)	<code>os.remove("a.txt")</code>

9. Hệ điều hành — module `os`

Hàm	Công dụng	Ví dụ
<code>import os</code>	Nạp thư viện os	<code>import os</code>
<code>os.getcwd()</code>	Lấy thư mục hiện tại	<code>os.getcwd()</code>
<code>os.listdir()</code>	Liệt kê file/thư mục	<code>os.listdir(".")</code>
<code>os.mkdir()</code>	Tạo thư mục mới	<code>os.mkdir("data")</code>
<code>os.rename()</code>	Đổi tên file/thư mục	<code>os.rename("a.txt", "b.txt")</code>
<code>os.path.exists()</code>	Kiểm tra tồn tại	<code>os.path.exists("a.txt")</code>

10. Ngày & giờ – module `datetime`

Hàm	Công dụng	Ví dụ
<code>import datetime</code>	Nạp thư viện	<code>import datetime</code>
<code>datetime.now()</code>	Lấy ngày giờ hiện tại	<code>datetime.datetime.now()</code>
<code>date.today()</code>	Lấy ngày hôm nay	<code>datetime.date.today()</code>
<code>strftime()</code>	Định dạng ngày giờ → chuỗi	<code>now.strftime("%d/%m/%Y")</code>
<code>strptime()</code>	Chuỗi → đối tượng ngày giờ	<code>datetime.datetime.strptime("2026-01-01", "%Y-%m-%d")</code>
<code>timedelta()</code>	Tính khoảng cách thời gian	<code>datetime.timedelta(days=7)</code>

11. Ngẫu nhiên — module `random`

Hàm	Công dụng	Ví dụ
<code>import random</code>	Nạp thư viện	<code>import random</code>
<code>random()</code>	Số thực ngẫu nhiên 0-1	<code>random.random()</code>
<code>randint()</code>	Số nguyên ngẫu nhiên trong khoảng	<code>random.randint(1,10)</code>
<code>uniform()</code>	Số thực ngẫu nhiên trong khoảng	<code>random.uniform(1,10)</code>
<code>choice()</code>	Chọn ngẫu nhiên 1 phần tử	<code>random.choice([1,2,3])</code>
<code>shuffle()</code>	Xáo trộn danh sách	<code>random.shuffle(my_list)</code>
<code>sample()</code>	Lấy ngẫu nhiên nhiều phần tử không trùng	<code>random.sample([1,2,3],2)</code>

12. Tiện ích cho hàm

Hàm	Công dụng	Ví dụ
<code>def</code>	Định nghĩa hàm	<code>def cong(a,b): return a+b</code>
<code>isinstance()</code>	Kiểm tra kiểu dữ liệu	<code>isinstance(5, int) → True</code>

Hàm	Công dụng	Ví dụ
<code>issubclass()</code>	Kiểm tra lớp con	<code>issubclass(bool, int) → True</code>
<code>hasattr()</code>	Kiểm tra đối tượng có thuộc tính không	<code>hasattr(obj, "name")</code>
<code>globals()</code>	Lấy các biến toàn cục	<code>globals()</code>
<code>vars()</code>	Lấy thuộc tính của đối tượng	<code>vars(obj)</code>

13. Hằng số dựng sẵn

Hằng số	Công dụng
<code>True / False</code>	Giá trị logic đúng/sai
<code>None</code>	Đại diện cho "không có giá trị"
<code>NotImplemented</code>	Dùng khi 1 phép toán chưa được hỗ trợ
<code>Ellipsis (...)</code>	Placeholder, thường dùng trong slicing nâng cao

14. Hàm nâng cao

Hàm	Công dụng	Ví dụ
<code>callable()</code>	Kiểm tra đối tượng có gọi được không (như hàm)	<code>callable(print) → True</code>

Hàm	Công dụng	Ví dụ
<code>compile()</code>	Biên dịch chuỗi code thành object thực thi	<code>compile("1+1", "<s>", "eval")</code>
<code>exec()</code>	Thực thi đoạn code dạng chuỗi	<code>exec("x=5")</code>
<code>eval()</code>	Tính giá trị biểu thức dạng chuỗi	<code>eval("2+3")</code> → 5
<code>lambda</code>	Tạo hàm ẩn danh (1 dòng)	<code>f = lambda x: x*2</code>

15. Xử lý lỗi (try/except)

Từ khóa	Công dụng	Ví dụ
<code>try</code>	Khối code có thể gây lỗi	<code>try: ...</code>
<code>except</code>	Xử lý khi có lỗi	<code>except ValueError: ...</code>
<code>raise</code>	Tự tạo ra lỗi	<code>raise ValueError("sai")</code>
<code>finally</code>	Luôn chạy dù có lỗi hay không	<code>finally: f.close()</code>
<code>assert</code>	Kiểm tra điều kiện, báo lỗi nếu sai	<code>assert x > 0</code>

16. Gỡ lỗi (debug)

Hàm	Công dụng	Ví dụ
<code>breakpoint()</code>	Dừng chương trình để debug	<code>breakpoint()</code>
<code>logging.debug()</code>	Ghi log mức debug	<code>logging.debug("chi tiết")</code>
<code>logging.info()</code>	Ghi log thông tin	<code>logging.info("đang chạy")</code>
<code>logging.error()</code>	Ghi log lỗi	<code>logging.error("có lỗi")</code>

17. Module

Hàm	Công dụng	Ví dụ
<code>import</code>	Nạp thư viện/module	<code>import math</code>
<code>dir()</code>	Liệt kê thuộc tính/hàm của đối tượng	<code>dir(str)</code>
<code>__import__()</code>	Nạp module theo tên dạng chuỗi	<code>__import__("os")</code>
<code>reload()</code> (từ <code>importlib</code>)	Nạp lại module đã sửa	<code>importlib.reload(math)</code>

18. Bộ nhớ (memory)

Hàm	Công dụng	Ví dụ
<code>memoryview()</code>	Xem dữ liệu nhị phân mà không copy	<code>memoryview(b"abc")</code>
<code>bytearray()</code>	Tạo mảng byte có thể thay đổi	<code>bytearray(5)</code>
<code>bytes()</code>	Tạo dữ liệu byte bất biến	<code>bytes("abc", "utf-8")</code>
<code>array()</code> (từ module <code>array</code>)	Tạo mảng kiểu dữ liệu cố định	<code>array.array('i', [1,2,3])</code>

Cách dùng bảng này

- Khi gặp code có hàm lạ, `Ctrl+F` tìm tên hàm trong bảng để tra nhanh công dụng + ví dụ.
- Chỉ cần thuộc nhóm **1, 2, 3, 4, 12, 15** trước — đây là nhóm dùng nhiều nhất khi mới học (theo lộ trình Tuần 1-2 của bạn). Các nhóm còn lại tra khi cần.